Oficina
Dossiê técnico da Fase 3

Arquitetura, componentes, sequências, modelo relacional, RFCs, ADRs e validação do ambiente de homologação.

Revisão de 31/10/2026 (BRT)

Este dossiê reúne a arquitetura e as evidências da validação em homologação. O PDF único de links para envio ao Portal está em arquivo próprio. A matriz detalha os testes e os limites do Learner Lab.

Vídeo incluído: YouTube (13min29s). Logs e traces demonstrados conforme confirmação do autor. A massa contém somente CPF e IDs gerados para demonstração. Nenhuma senha, token ou credencial é incluída.

Comece aqui - demonstração da Fase 3

> **Ambiente encerrado:** a homologação foi validada e os recursos AWS foram removidos a pedido do responsável. Os links de API/Grafana e os IDs da massa são referências da demonstração. Veja o registro de encerramento; para repetir as chamadas, reimplante o ambiente e execute o seed novamente.

Acessos rápidos

Acesso	Link
Swagger	Abrir
Grafana	Abrir
Dashboard de infraestrutura / HPA	Abrir
Action de seed automático	Abrir
Seed anterior aprovado	Abrir
Deploy aprovado com smokes	Abrir
PR integrado com CI aprovado	Abrir
PDFs individuais	Abrir
Roteiro	Abrir

Endpoints dependem do Learner Lab ativo. Grafana usa usuário admin e a senha configurada para ele. A senha da API é APP_SEED_ADMIN_PASSWORD de homolog; são acessos separados.

A massa já é preparada pelo seed

Não é necessário cadastrar cliente, veículo ou peça no Postman. Antes da demonstração, o seed automático cria um cliente ativo com CPF gerado para teste, um veículo, uma peça com estoque e seis OS, uma em cada estado. Os dados são sintéticos e o e-mail usa example.invalid. O SMTP é capturado pelo sandbox do Mailtrap.

O seed aprovado exporta FIAP-Massa-Pronta.postman_environment.json no artifact da Action. Esse ambiente contém o CPF e os IDs sintéticos prontos para uso. Senha e tokens ficam vazios. O Summary mantém apenas contagens. O modo load_only não gera massa. Cada seed cria novo lote e deixa os registros em homologação; não execute novamente só para abrir outra OS.

A massa desta revisão já está pronta

Importar diretamente o ambiente preenchido e a collection curta. Dados detalhados da massa e execução aprovada. Basta preencher a senha. O passo de seed abaixo só é necessário para renovar a massa.

Preparar uma vez ou renovar a massa

1. Abrir a Action de seed, Run workflow, branch main, operation=seed_only e work_orders=6.

Os campos de duração/concorrência não são usados nesse modo. Aguardar conclusão verde.

2. No fim da página da execução, abrir Artifacts e baixar homolog-simulation-<run_id>.

Importar o arquivo FIAP-Massa-Pronta.postman_environment.json no Postman.

3. Importar a collection de demonstração.

Selecionar o ambiente Massa Pronta e preencher apenas adminPassword localmente.

4. O ambiente desta revisão já está publicado nesta pasta. Se o Lab/banco for recriado, substituir pelo artifact de um novo seed.

O seed anterior aprovado, vinculado na tabela, não exportava o ambiente Postman. A execução nova aprovada já exporta o ambiente.

Executar na gravação - somente 12 requests

No Runner: uma iteração, Delay=6000ms, sem CSV, sem paralelismo. Ou enviar manualmente em ordem, esperando seis segundos entre as operações de OS. Não é teste de carga.

Request	Demonstração	Esperado
01	Login do funcionário, salva adminToken	200
02	Abre OS com cliente/veículo do seed, salva workOrderId	201 / RECEIVED
03	Inicia diagnóstico	200 / DIAGNOSIS
04	Gera orçamento usando a peça do seed, salva estimateId	201 / WAITING_APPROVAL
05	Login por CPF da massa, salva customerToken	200
06	Cliente consulta suas OS	200
07	Mesma consulta sem token	401
08	Cliente tenta rota operacional	403
09	Cliente aprova o orçamento	200 / IN_PROGRESS
10	Funcionário conclui por valor final 50	200 / COMPLETED
11	Registra entrega	200 / DELIVERED
12	Consulta a OS e verifica estado final	200 / DELIVERED

A collection salva IDs/tokens automaticamente e envia Content-Type application/json nos PATCH, inclusive sem corpo. Pode repetir o ciclo sem recriar a massa: cada iteração abre uma nova OS e usa uma unidade da peça. O seed cria estoque de work_orders+10 (16 no lote padrão de seis OS), mas parte é reservada/consumida pelas próprias OS do seed; não é estoque infinito. Se esgotar, preparar outro lote pela Action.

Em falha inesperada, o Runner para. Não há retry automático de escrita: um 503 pode ter ocorrido depois de persistir a operação. Conferir o resultado antes de repetir. Tokens expirados são renovados repetindo o login; senha/tokens não são publicados no repositório.

Swagger e Postman

Swagger é o catálogo para explorar os endpoints da aplicação com Try it out. A collection é o caminho curto para a apresentação, sem cadastros manuais. O login CPF /v1/auth/customer é da Lambda e pode não aparecer no OpenAPI da aplicação; está incluído na collection. A collection não pretende conter todo CRUD. Sua validação estática não substitui uma execução real pelo usuário com a senha local.

Seed e carga são coisas diferentes

O padrão da Action é seed_only: prepara dados, sem carga de CPU. Para demonstrar HPA, selecionar load_only separadamente, duração 180 segundos, concorrência 2, teto global de quatro requisições/s. O workflow observa o HPA, sem forçar escala. O perfil leve ainda precisa de validação de estabilidade. Summary e hpa.csv registram as evidências.

Já houve escala 2 -> 4 -> 2, mas a carga anterior teve oito 503 em 211 tentativas. Não apresentar esse teste como aprovado. A investigação permanece aberta. Traces, métricas HTTP/negócio e alertas específicos de OS continuam com pendências na matriz.

O PR 86 mostra checks aprovados e merge de uma mudança pequena na main; não é deploy. Para demonstrar entrega automatizada, usar o run homolog aprovado da tabela.

O Portal pede vídeo de até 15 minutos e PDF único com links. Os documentos e diagramas individuais ficam neste repositório e não dependem de ZIP.

Manutenção da massa e acesso local

Action de manutenção: senha e limpeza de lote.

- preview_seed é o padrão: informar o batch FIAP-DEMO-xxxxxxxxxxxx da página Massa Pronta.

Mostra contagens e não apaga nada.

- delete_seed exige batch e confirmation com exatamente o mesmo valor. Apaga somente

aquele lote e suas dependências, em transação. Não existe opção de limpar todas as massas.

- Cliente do teste Authorizer, estoque/veículo referenciado fora do lote e dados que não

correspondam à massa sintética bloqueiam a exclusão. Falhas causam rollback.

- Após excluir o lote usado no ambiente Postman, executar seed_only e importar o ambiente

do lote novo. Os IDs antigos deixam de funcionar. Exclusão não possui backup automático.

- rotate_admin aplica ao administrador existente a senha da secret APP_SEED_ADMIN_PASSWORD

do Environment homolog e verifica o login. Trocar somente a secret não troca senha já cadastrada no banco; o seed inicial preserva usuários existentes.

O arquivo pessoal de acesso e o ambiente com senha preenchida ficam somente na máquina local do operador, fora do Git. Os arquivos publicados continuam com senha e tokens vazios.

Massa pronta - criada automaticamente

> **Ambiente encerrado:** a homologação foi validada e os recursos AWS foram removidos a pedido do responsável. Os links de API/Grafana e os IDs da massa são referências da demonstração. Veja o registro de encerramento; para repetir as chamadas, reimplante o ambiente e execute o seed novamente.

Seed aprovado em 31/10/2026, às 12h02 (America/Sao_Paulo). Execução e artifact.

A massa já existe em homologação. Não cadastrar cliente, veículo ou peça na gravação. Importar a collection e o ambiente preenchido, selecionar o ambiente no Postman e preencher apenas adminPassword localmente.

Dados sintéticos disponíveis

Campo	Valor
demoName	FIAP-DEMO-3dd67630cefa
demoCpf	56723007158
customerId	d8177e01-df1c-4903-b713-f92a78c6ba85
vehicleId	ff8d6372-8db3-4c13-9492-eba80470a6fa
partId	dcfffbd1-b8a9-4fc5-aac6-8190ffdecbba

Esses valores pertencem somente ao lote de demonstração criado pelo seed. O CPF foi gerado para teste e o e-mail usa example.invalid. IDs são referências para a collection, não segredos. Nenhuma senha, token JWT ou credencial AWS foi incluída.

O que o seed confirmou

- Um cliente, um veículo, uma peça e seis OS criados.

- Uma OS em cada estado: RECEIVED, DIAGNOSIS, WAITING_APPROVAL, IN_PROGRESS, COMPLETED e DELIVERED.

- Vinte respostas HTTP 200 e treze HTTP 201; nenhum 503 nesta execução.

- O estoque inicial da peça foi 16; as OS do seed já reservam/consomem parte dele.

As seis OS existentes servem para explorar os estados. A collection curta abre outra OS com os mesmos cadastros e percorre todo o ciclo, salvando workOrderId e estimateId. Não é necessário copiar os IDs manualmente: já estão no ambiente importável. Se o banco/Lab for recriado ou a massa removida, rodar seed_only novamente e importar o ambiente do novo artifact. Os registros não são apagados automaticamente.

Acessos de demonstração

API: adminUsername=admin; adminPassword deve receber a senha APP_SEED_ADMIN_PASSWORD do ambiente homolog no seu Postman. O login do cliente usa demoCpf e recebe um JWT automaticamente. Para Grafana, usuário admin e sua senha própria.

Esta execução comprova o seed. A validação posterior de carga, indicadores de negócio, traces e alertas está na matriz atualizada.

Fase 3 - documentação da Oficina

> **Ambiente encerrado:** a homologação foi validada e os recursos AWS foram removidos a pedido do responsável. Os links de API/Grafana e os IDs da massa são referências da demonstração. Veja o registro de encerramento; para repetir as chamadas, reimplante o ambiente e execute o seed novamente.

Pacote arquitetural e roteiro de homologação. Revisão: 31/10/2026 (BRT). O estado descrito é o ambiente acadêmico implantado. A matriz de validação diferencia implementação, evidência observada e pendências; documentação não substitui um teste operacional.

PDF para enviar à instituição

Documento único de entrega ao Portal: versão para submissão, com os quatro repositórios, documentação, confirmação do avaliador e vídeo de 13min29s. A matriz de validação apresenta os testes e limites observados. Como atualizar a entrega.

Acesso rápido

Comece aqui: Postman, massas, seed, Grafana, PR e deploy.

Baixar PDFs individuais e consultar diagramas em diagramas. Não é necessário baixar ZIP.

Massa já cadastrada e ambiente Postman.

Leitura e apresentação

1. Arquitetura, componentes e fronteiras.
2. Sequências de autenticação e abertura de OS.
3. Modelo relacional, justificativa e evolução.
4. Matriz dos requisitos e evidências.
5. Roteiro do vídeo de até 15 minutos.
6. Folha de entrega e links.

Decisões

Documento	Tema
RFC-001	AWS, EKS e serviços gerenciados no Learner Lab
RFC-002	Escolha formal do PostgreSQL e consistência
RFC-003	CPF, JWT RS256, Gateway e RBAC
ADR-001	HTTP síncrono e fronteiras de acesso
ADR-002	HPA e diferença entre pods e nós
ADR-003	Stack compartilhada, dois dashboards e armazenamento efêmero

Os RFCs formalizam a justificativa da solução existente e registram os pontos em aberto. Os ADRs documentam decisões materializadas no código; não simulam aprovação histórica da equipe. Os ADRs existentes da aplicação continuam válidos nos respectivos escopos.

Fontes e propriedade

- Aplicação: Quarkus, domínio, REST, migrations e manifests da aplicação.

- Autenticação: funções Go e Terraform correspondente.

- Kubernetes: EKS, Gateway, VPC Link e observabilidade.

- Banco: RDS e controles de rede do banco.

Diagramas Mermaid estão versionados em `diagramas/*.mmd`; as imagens SVG/PNG permitem apresentação fora do GitHub. O modelo foi confrontado com entidades/migrations, sem consultar tabelas nem dados de clientes. Nenhuma credencial ou dado de cliente integra este pacote.

Dossiê para leitura offline

PDF técnico desta revisão. Os textos usam A4; os quatro diagramas usam páginas A3 para preservar a legibilidade. O dossiê complementa o PDF único de links exigido para o Portal; o vídeo já está incluído na entrega.

Para reproduzir a partir da raiz do repositório, usar Python com Pillow/ReportLab e Graphviz: `python3 docs/fase3/gerar-diagramas.py` e `python3 docs/fase3/gerar-dossie.py`.

Arquivos separados

Os PDFs individuais ficam em `output/pdf/separados/`, gerados por `python3 docs/fase3/gerar-separados.py`. Incluem arquitetura, sequências, modelo de dados, cada RFC/ADR, matriz, roteiro simples e folha de entrega. Diagramas PNG/SVG e fontes Mermaid continuam em `docs/fase3/diagramas/`. Os PDFs e fontes editáveis estão publicados individualmente neste repositório. O enunciado continua exigindo vídeo e PDF único de links para o Portal.

Arquitetura e componentes

Escopo e estado

A solução organiza atendimento, clientes, veículos, ordens de serviço, orçamentos, serviços e peças. O ambiente confirmado é homologação em AWS us-east-1 no Learner Lab. Os quatro repositórios possuem workflows e regras de PR em main. Os workflows de produção estão definidos; não foi encontrada execução de produção nesta revisão.

Caminhos de execução

Origem	Destino	Contrato e responsabilidade
Cliente externo	API Gateway HTTP	HTTPS; entrada pública, roteamento e integração com Authorizer
POST /v1/auth/customer	Lambda customer-auth	Valida CPF, consulta existência/status no PostgreSQL e emite JWT
Rotas protegidas	Lambda jwt-authorizer	Verifica JWT; REQUEST, payload 2.0, resposta simples e cache zero
Gateway	VPC Link -> NLB interno -> aplicação	HTTP privado até a API em 8080; a aplicação aplica RBAC e regras de negócio
Gateway /observability	VPC Link -> NLB interno -> Grafana	Login próprio do Grafana; a rota não usa o JWT da aplicação
API / Lambda customer-auth	RDS PostgreSQL 16	Banco privado, TLS; consultas de autenticação e persistência de domínio
Prometheus	Service de management da API	ClusterIP em 9000; métricas /q/metrics; sem exposição pública desse listener
Blackbox	Gateway /q/swagger-ui/	Sondagem pública periódica; mede disponibilidade dessa rota, não todas as regras de negócio
Alloy	Loki / Tempo	Logs dos pods e OTLP; serviços internos
Lambdas / Gateway	CloudWatch	Logs operacionais; tracing OTLP da Lambda é opcional e não foi ativado por esta documentação

As rotas OpenAPI/Swagger de homologação são exceções públicas explícitas. Management, Prometheus, Alertmanager, Loki, Tempo e receptor OTLP não possuem rota pública própria. Os NLBs de aplicação e Grafana são internos; o ELB público legado já estava ausente na revisão.

Responsabilidade de cada repositório

Repositório	Recursos e entrega
oficina-app	Java 21, Quarkus, API, domínio hexagonal, Dockerfile, imagem ECR, manifests/Kustomize, rollout, smoke e contrato upstream
oficina-lambda-auth	Go, dois binários Lambda, autenticação CPF, validação JWT, pacote ZIP, publicação de versão e contrato authorizer
oficina-infra-k8s	Terraform de EKS/ECR/Gateway/VPC Link; stack Helm; listener Grafana e reconciliação das rotas
oficina-infra-db	Terraform de RDS PostgreSQL e rede; state independente por ambiente

Entrega e convergência

PRs para main/homolog/production passam pelas verificações definidas por cada repositório. Push em homolog/production aciona os workflows de entrega correspondentes. A stack de observabilidade usa disparo manual serializado porque é compartilhada pelo cluster.

A aplicação publica `upstream.json`; a Lambda publica `authorizer.json`; observabilidade publica `grafana.json`. Os contratos ficam em S3 e contêm referências de integração e SHA, sem valores de secrets. A reconciliação preserva o outro ambiente e recusa planos fora dos recursos de roteamento autorizados. Terraform usa plano salvo e lock no backend S3. O bucket/região da publicação são transmitidos por outputs para o workflow reutilizável, evita-se resolver variáveis de outro escopo na chamada.

Segurança e limites reais

- O Gateway valida identidade; a API controla papel e acesso a cada recurso. CUSTOMER não recebe acesso operacional apenas por possuir JWT válido.

- CPF é um identificador de conhecimento e não um segundo fator. A solução atende ao fluxo acadêmico; operação comercial exige avaliar autenticação adicional, abuso e revogação.

- Segredos são fornecidos por GitHub Environments/Organization e montados/injetados nos workloads. State Terraform pode conter material sensível e não deve ser publicado.

- RDS é privado. O cliente reativo da API usa TLS com validação da cadeia relaxada no Lab; uma implantação comercial deve validar a CA e o hostname.

- ClusterIP não é isolamento entre pods. Não foi comprovado enforcement de NetworkPolicy nesta revisão.

- Homolog/production compartilham cluster e stack de monitoramento; namespaces não equivalem a isolamento físico.

- RDS Single-AZ, HPA mínimo de uma réplica e armazenamento efêmero de telemetria limitam disponibilidade/durabilidade. Não se afirma alta disponibilidade corporativa.

- O HPA escala pods. Não há comprovação de Cluster Autoscaler para criação automática de nós.

Base técnica conferida

- Aplicação: `f56897c0a4959d1ed3576d15ccc1920d9bd53e24` (homolog).

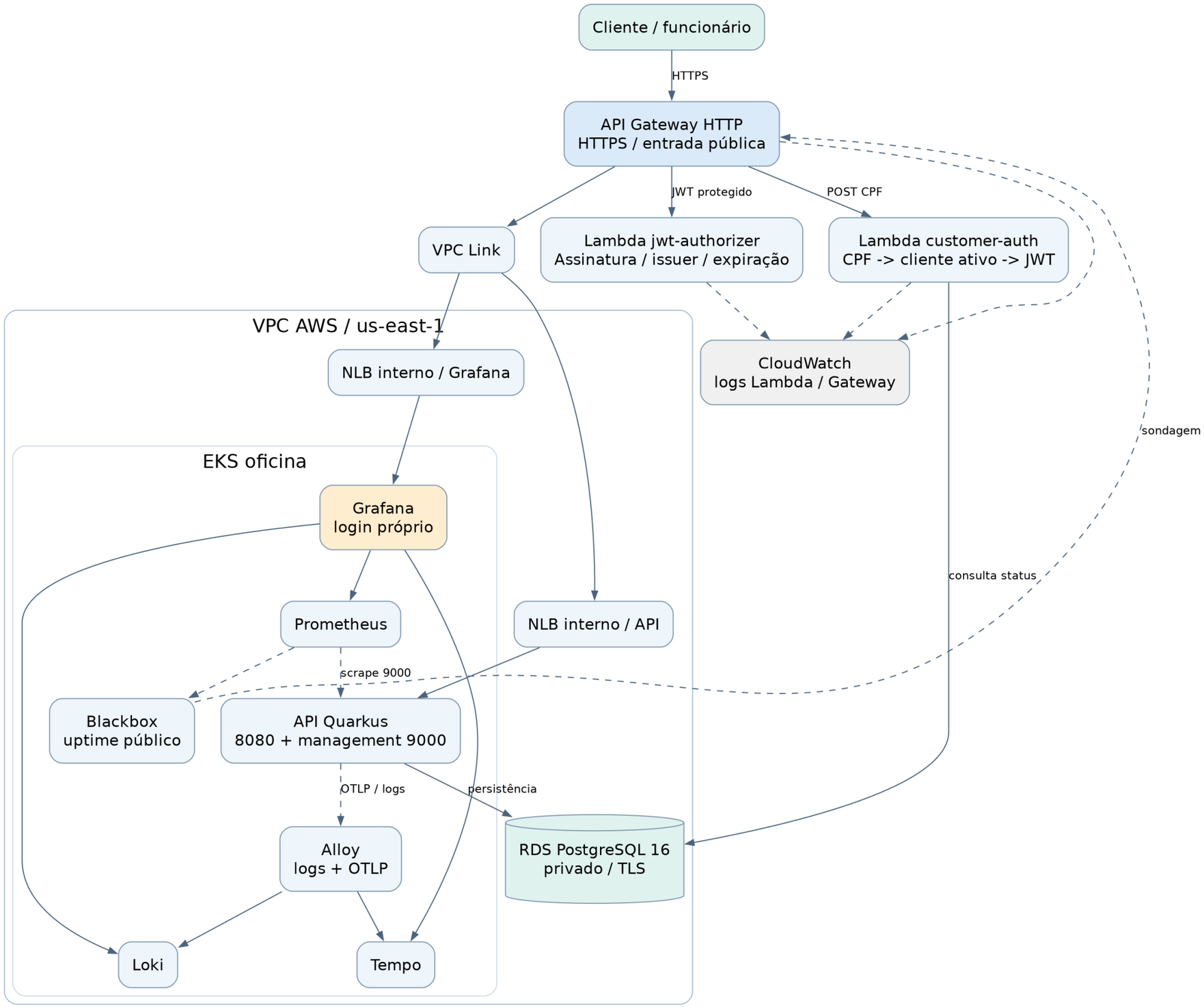
- Lambda: `8c0d4320eafeeb5b242020e76bfeee4be49f66e4` (homolog).

- Banco: `657489d3d6a932813c41835ec78d79b2bfac8838` (homolog).

- Kubernetes/observabilidade: PRs 26-29 e workflows registrados na matriz de validação.

Referências verificáveis: API, Gateway, Lambda, RDS.

Componentes da solução



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Sequências dos fluxos obrigatórios

CPF, emissão de token e consumo protegido

1. O cliente envia o CPF à rota pública POST /v1/auth/customer via TLS.

2. O Gateway integra essa rota com a Lambda customer-auth, sem exigir um token prévio.

3. A função valida o documento e consulta o cliente no PostgreSQL privado.

4. Ausência/inatividade impedem emissão. Com cliente ativo, emite JWT RS256 com issuer,

expiração, identidade e grupo CUSTOMER. Nenhum valor de CPF/JWT deve aparecer no vídeo.

5. Ao chamar GET /v1/me/work-orders, o cliente envia Authorization Bearer.

6. O Gateway pode rejeitar ausência de identidade com 401 sem invocar a função. Com

identidade presente, a Lambda Authorizer valida o token e responde permitir/negar.

7. A aplicação valida o contexto de segurança e o vínculo do recurso ao cliente.

O Authorizer não consulta o banco a cada requisição. Desativação de cliente não equivale a revogação instantânea de tokens já emitidos. O cache do Authorizer está em zero, mas isso não altera a validade criptográfica/temporal do JWT.

Abertura da ordem de serviço

Pré-condição: funcionário autenticado com papel ADMIN ou MECHANIC. O login de funcionário é POST /v1/auth/login na aplicação. O JWT CUSTOMER do fluxo anterior não autoriza este POST.

1. O funcionário envia POST /v1/work-orders com Bearer e referências de cliente/veículo.

2. O Gateway invoca o Authorizer; token recusado encerra o fluxo sem executar o caso de uso.

3. A aplicação aplica RBAC e valida o DTO. Identidade aceita pelo Gateway não substitui RBAC.

4. WorkOrderService.create executa com @WithTransaction. Consulta cliente/veículo e o

funcionário atribuído quando fornecido, e abre a OS em RECEIVED.

5. Quando existem serviços/peças iniciais, resolve catálogo, cria linhas/orçamento e

registra a evolução correspondente. Sem solicitação inicial, a OS não cria orçamento.

6. O adaptador de notificação prepara o acompanhamento via SMTP. Falhas de entrega são

tratadas pelo adaptador; não há Lambda de notificações implantada nesta arquitetura.

7. A transação é confirmada antes de a resposta HTTP 201 ser observada pelo cliente,

incluindo Location. Falhas transacionais não devem produzir uma OS parcial.

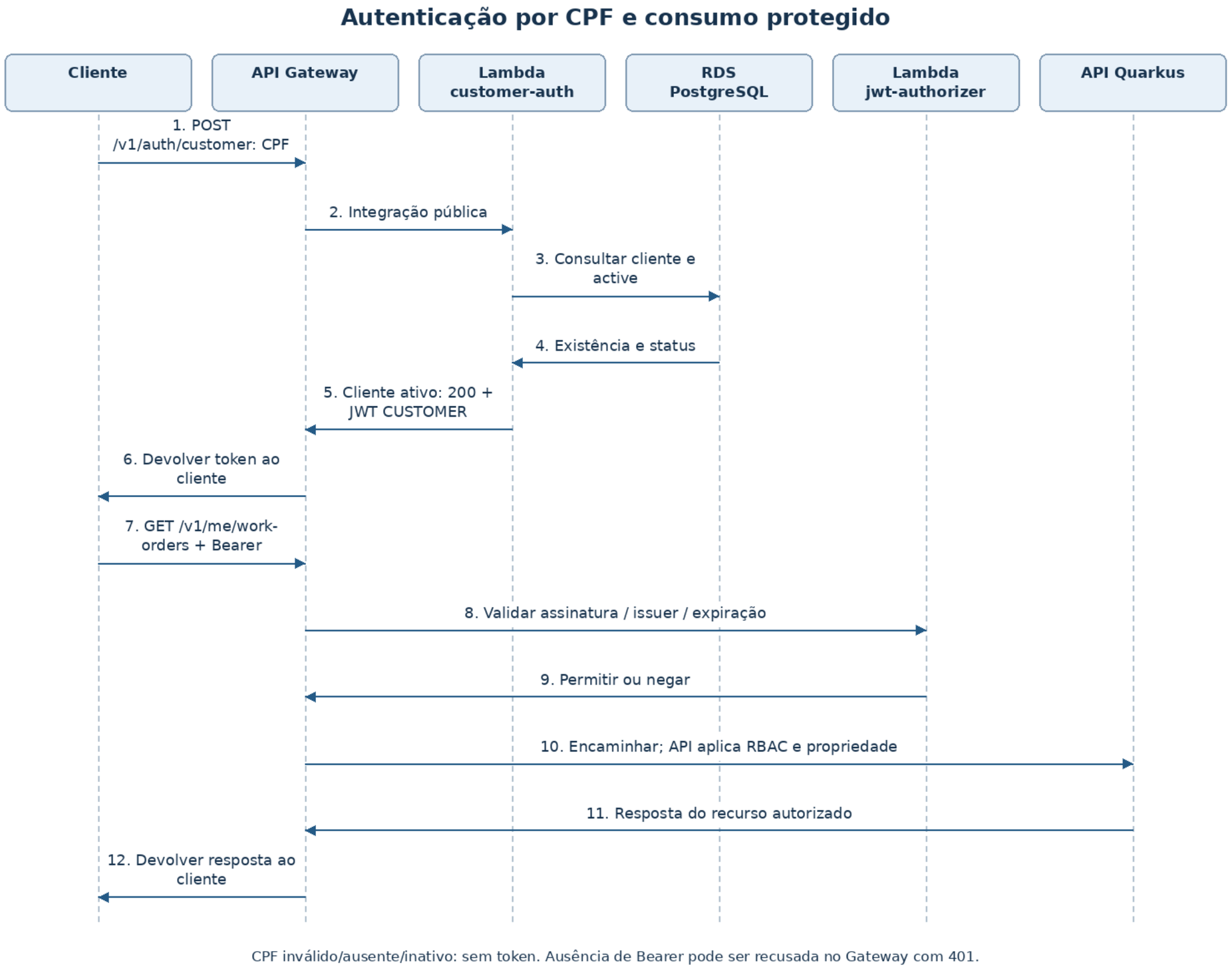
A chamada de notificação ocorre no fluxo reativo do caso de uso; este diagrama não promete padrão outbox, fila, entrega exatamente uma vez nem commit anterior ao envio SMTP.

Diagramas editáveis

Os arquivos `.mmd` correspondentes incluem os caminhos alternativos de erro e são renderizáveis no GitHub/Mermaid. As imagens exportadas destacam o caminho de sucesso e as recusas essenciais.

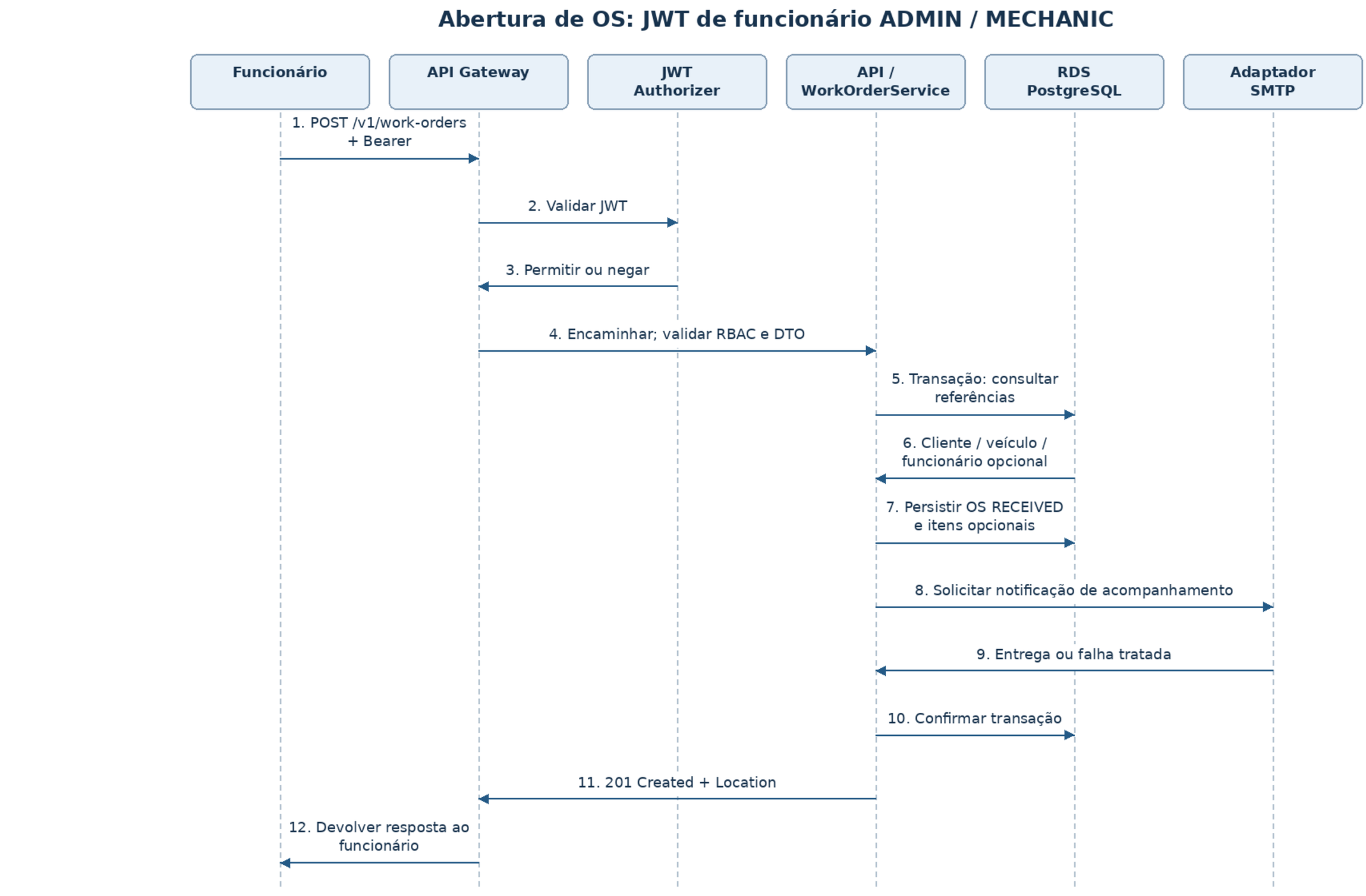
Fonte: WorkOrderController, WorkOrderService e handler CPF.

Autenticação CPF



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Abertura de OS



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

Banco de dados e modelo relacional

Justificativa formal do PostgreSQL gerenciado

A Oficina mantém relações fortes entre cliente, veículo, OS, orçamento, itens e estoque. A abertura de OS e a reserva/devolução de peças exigem transações e integridade entre entidades. PostgreSQL fornece transações ACID, chaves/constraints e consultas relacionais compatíveis com Hibernate Reactive/Panache, JDBC/Flyway e o driver pgx da Lambda. RDS delega operação do mecanismo e backups configurados ao serviço gerenciado.

Alternativas consideradas: MySQL gerenciado também atende relações/transações, mas exigiria adaptar tipos, drivers e migrations sem benefício medido para este projeto. Um banco NoSQL exigiria redesenhar as consultas e a consistência entre agregados. Não foi realizado benchmark comparativo; a escolha é de adequação funcional e continuidade da stack, não superioridade de desempenho.

Modelo implementado

Tabela	Identidade e relacionamentos essenciais
customer	UUID; documento único; active controla elegibilidade no login CPF
vehicle	UUID; placa única; associação a customer (nullable no mapeamento atual)
worker	UUID; perfil, email único, password_hash e active
work_order	UUID; cliente e veículo obrigatórios; funcionário opcional; status e datas do ciclo
work_order_history	UUID; OS obrigatória; status anterior/novo e instante da transição
estimate	UUID; OS obrigatória; status e valores de peças/mão de obra/total
estimate_item	UUID; orçamento e peça obrigatórios; quantidade, preço e total
parts	UUID; quantidade de estoque, mínimo e @Version para concorrência otimista
service_item	UUID; catálogo, preço base, duração estimada e active
work_order_service	UUID; OS obrigatória; catálogo opcional; descrição/preço registrados
estimate_decision_token	UUID atribuído pelo domínio; IDs de OS/orçamento, decisão e validade/consumo
app_users	Identificador numérico; username único, credencial, role e active; entidade de autenticação separada

As relações JPA @ManyToOne são relações mapeadas, sujeitas ao DDL efetivamente aplicado. `estimate_decision_token.work_order_id/estimate_id` são campos UUID, não associações JPA: o diagrama usa linhas tracejadas para vínculos lógicos e não afirma existência de FKs físicas. `app_users` possui entidade JPA no código atual; a documentação legada que o chamava de tabela sem entidade precisa ser substituída por este retrato atualizado. Valores de credenciais e dados pessoais não integram o diagrama.

Cardinalidades e consistência

- Cliente pode ter vários veículos/OS; OS pertence a um cliente e veículo.

- Funcionário pode atender várias OS, e sua atribuição é opcional.

- OS possui histórico, serviços e orçamentos. A modelagem não impõe 1:1 entre OS e orçamento.

- Orçamento contém itens que referenciam peças. Valores monetários usam BigDecimal/decimal.

- Serviço realizado pode manter a referência do catálogo ou apenas o retrato de descrição/preço.

- O estoque usa versão otimista. A consistência do fluxo também depende dos casos de uso e transações.

- Há constraints para estados canônicos em work_order/work_order_history; estados históricos incompatíveis são tratados pela migration V2, preservando campos legados.

Evolução do schema e pontos de revisão

V1 cria o schema; V2 ajusta o ciclo da OS e seu histórico; V3 adiciona active em customer. Flyway migrate-at-start está habilitado, mas Hibernate schema-management.strategy=update continua configurado fora dos testes. Logo, Flyway ainda não é a única autoridade do DDL. A revisão não executou DDL nem consultou dados no banco.

A melhoria recomendada é uma baseline completa e migrations explícitas para criação/evolução, seguida de validate no runtime. Isso exige planejar migração do banco já existente; não foi aplicado automaticamente. Não se afirma que PK/FK/índices do schema vivo foram inspecionados.

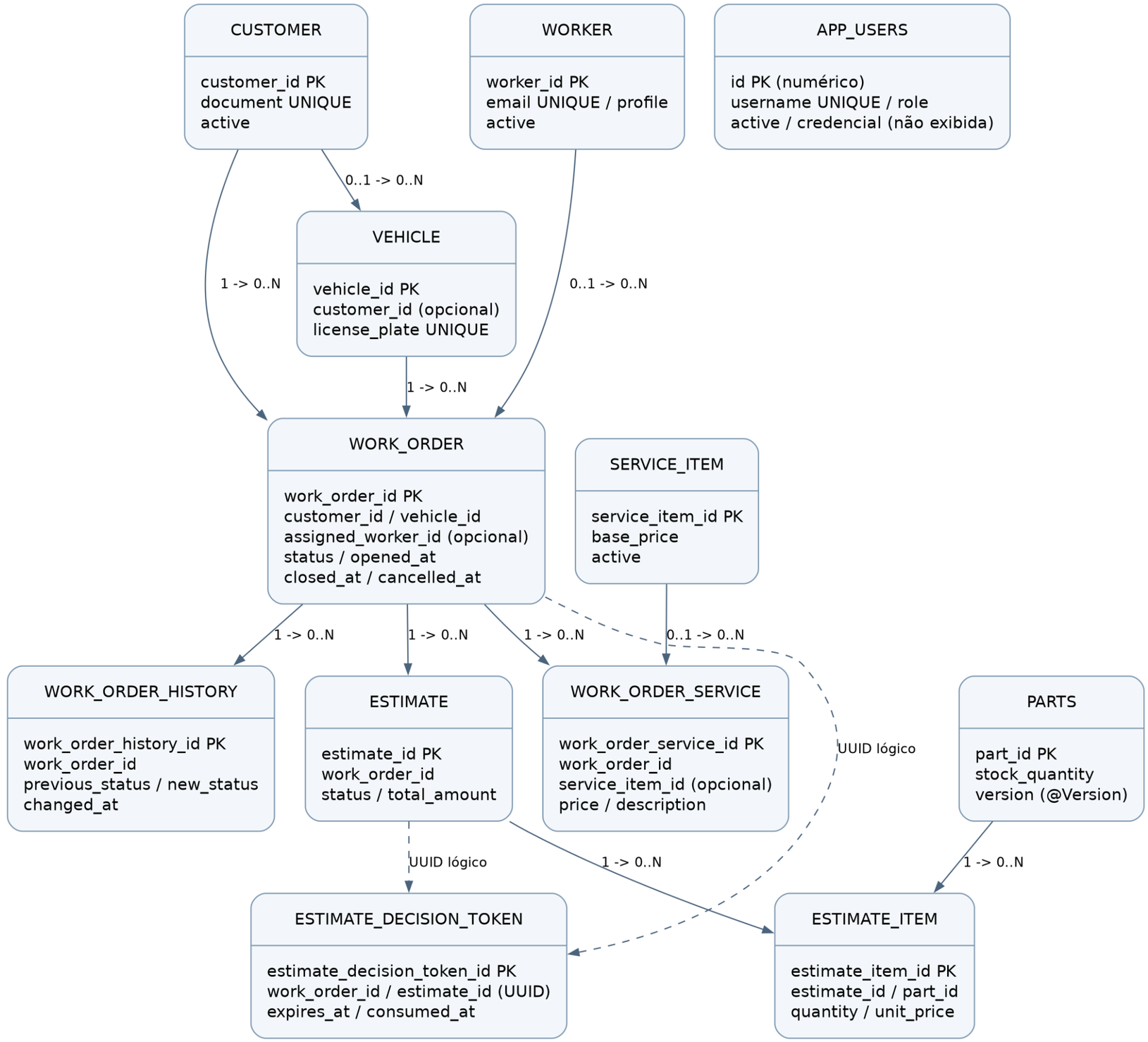
Performance e indicadores de negócio

Documento único/placa/username/email de funcionário têm unicidade no mapeamento. Consultas por cliente, fila operacional, datas e histórico precisam de índices avaliados por planos reais antes de otimizar. Esta documentação não inventa índices nem resultados EXPLAIN.

O endpoint atual de tempo médio calcula abertura->fechamento das OS concluídas. Isso não entrega volume por dia nem médias separadas de Diagnóstico/Execução/Finalização no Grafana. Essas agregações devem usar dados persistidos e históricos de transição, definir timezone, tratamento de cancelamentos/reentrada e semântica de Finalização. Contadores apenas na memória de um pod não substituem agregação correta com réplicas/reinícios.

Fonte: entidades de persistência da aplicação e migrations.

Modelo relacional



Fonte editável e versão vetorial no diretório docs/fase3/diagramas. Diagrama de homologação; limites e caminhos alternativos no texto.

RFC-001 - AWS e serviços gerenciados

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Problema

O desafio exige Gateway, função serverless, Kubernetes escalável, banco gerenciado e IaC. O Learner Lab limita IAM e oferece credenciais temporárias. A solução precisa ser demonstrável sem esconder essas restrições nem criar dependência de acesso administrativo não disponível.

Solução adotada

AWS API Gateway HTTP, Lambda Go, EKS, ECR, RDS PostgreSQL, S3 para state e CloudWatch. Terraform provisiona recursos AWS; Helm instala observabilidade depois que o cluster está acessível. Quatro repositórios mantêm fronteiras de entrega e workflows por ambiente.

Alternativas e critérios

Alternativa	Avaliação
EKS + RDS + Lambda	Alinha os requisitos e aproveita o Lab; exige gerir compatibilidade Kubernetes e limites IAM
Kubernetes autogerido + banco em VM	Aumenta operação e banco em VM não atende diretamente o requisito de banco gerenciado
Gateway/autenticação no pod	Possível tecnologicamente, mas mantém acoplamento de disponibilidade e não substitui a função serverless exigida
Outra nuvem	Viável pelo enunciado; exigiria reprovisionamento e credenciais fora do ambiente disponibilizado

Consequências

Roles preexistentes do Lab restringem privilégio fino. Homolog e produção compartilham EKS; RDS é por ambiente. O ambiente não promete tolerância a falha de região/AZ. Custos dependem principalmente de EKS, nós, RDS e dois NLBs internos; não foi efetuado cálculo de custo mensal.

Critérios de aceitação: apply revisado, workloads prontos, tráfego via Gateway, testes de negação de acesso e evidência de monitoramento. Evolução comercial exige IAM dedicado, isolamento e HA dimensionados por requisitos medidos.

RFC-002 - PostgreSQL gerenciado e evolução de dados

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Decisão e critérios

Adotar PostgreSQL 16 em Amazon RDS. As operações do domínio requerem transações, relacionamentos e agregações; a stack já utiliza Hibernate Reactive, JDBC/Flyway e pgx. RDS atende ao requisito explícito de banco gerenciado. Ver modelo e justificativa.

Alternativas

MySQL gerenciado é funcionalmente possível, porém acrescenta migração de dialeto/driver e não há evidência de ganho neste sistema. NoSQL não elimina a necessidade de consistência entre OS, orçamento e estoque, e exigiria redesenho do modelo. Banco em container não atende à intenção de serviço gerenciado nem transfere operação/backups à nuvem.

Integridade e operação

Preservar UUIDs do domínio, decimal para dinheiro, unicidade onde mapeada, estados canônicos e transações do caso de uso. O banco é privado, com transporte TLS. Backups seguem Terraform de cada ambiente; Multi-AZ é desativado no Lab. Schema e credenciais não devem ser colocados em artefatos públicos.

Pontos em aberto

DDL ainda combina Flyway e schema-management=update. A baseline completa e a validação exclusiva por migrations são uma evolução planejada. Índices devem ser justificados por consulta/plano medido; esta revisão não fez acesso SQL. Definir métricas diárias e duração por etapa a partir do histórico antes de afirmar conformidade integral de observabilidade.

RFC-003 - CPF, JWT RS256 e autorização

Status: justificativa técnica da implementação existente, aberta à revisão da equipe.

Solução

A Lambda customer-auth valida o CPF e o status do cliente no RDS antes de emitir JWT. A Lambda jwt-authorizer valida assinatura/issuer/expiração para as rotas protegidas do HTTP API. A aplicação aplica RBAC e propriedade dos recursos. As funções são distintas: emitir uma identidade e aceitar uma identidade são responsabilidades diferentes.

Alternativas consideradas

JWT apenas na API simplifica o caminho interno, mas não demonstra o Authorizer no Gateway. Tokens opacos com sessão central permitem revogação mais imediata, mas adicionam consulta de sessão ao caminho de autenticação. Um provedor de identidade gerenciado pode melhorar segurança de uso comercial; não substitui automaticamente a função CPF exigida no desafio.

Consequências e limites

RS256 permite validar com chave pública sem distribuir a privada ao Authorizer. As chaves privadas continuam necessárias aos emissores e devem permanecer em secrets. Cache zero no Authorizer simplifica os testes de autorização e evita cache de uma decisão anterior. A emissão via CPF sem fator adicional é uma escolha acadêmica, insuficiente como única prova forte de identidade em um produto comercial.

Cliente inativo não recebe novo token. Um token já emitido pode continuar válido até expirar; não se afirma revogação imediata. CPF/JWT/senhas não devem ser exibidos no vídeo/logs.

Verificação

Matriz real em homolog: login conhecido e acesso CUSTOMER 200; documento sem cadastro e ausência de token 401; Bearer malformado, token alterado e rota operacional CUSTOMER 403. O status HTTP de negócio deve ser distinguido de falha 5xx de integração.

ADR-001 - Comunicação síncrona e fronteiras de autorização

Status: estado implementado documentado nesta revisão.

Contexto

O usuário precisa obter respostas da API e executar operações transacionais. Gateway, autenticação, RBAC e regras de negócio têm responsabilidades separadas.

Decisão registrada

Manter HTTP síncrono: HTTPS no Gateway, integrações Lambda para autenticação/Authorizer e VPC Link/NLB interno para a API. O código da aplicação usa portas/adaptadores e fluxo reativo. A autorização criptográfica do Gateway precede RBAC e propriedade de recurso na API. Management é um Service separado, sem rota pública. Grafana usa sua própria autenticação.

Consequências

O cliente observa imediatamente o status da operação. Falhas de rede/backend precisam de correlação e monitoramento. Não há contrato de fila/outbox/exatamente-uma-vez. SMTP é uma integração do adaptador da API e sua falha não deve ser confundida com erro de autenticação. Tokens, CPF e senhas ficam fora dos logs/documentos. Endpoints públicos são exceções explícitas.

Evidência

api-gateway.tf, scripts de smoke, controllers @RolesAllowed e portas/adaptadores da aplicação.

ADR-002 - HPA de CPU e memória

Status: estado implementado documentado nesta revisão.

Contexto

A API precisa aumentar capacidade em picos sem depender de intervenção por requisição. O Lab tem capacidade finita e a stack de observabilidade compartilha os nós.

Decisão registrada

Usar autoscaling/v2 para oficina-mecanica: mínimo 1 e máximo 6 réplicas; alvo CPU 60% e memória 80%. Metrics Server fornece recursos. Scale-up permite 2 pods a cada 15 segundos, sem janela inicial; scale-down estabiliza por 60 segundos e remove 1 pod por 30 segundos. Deployment usa rollout com maxUnavailable=0 e maxSurge=1, além de probes de saúde.

Consequências

HPA escala pods, não nós. O node group é gerido por Terraform; este ADR não afirma Cluster Autoscaler. Uma réplica mínima é economia acadêmica, sem garantir alta disponibilidade. O teto de 6 é um limite, não comprovação de que todas as réplicas cabem após a instalação de Prometheus/Loki/Tempo. Medir requests, limites, scheduling e carga antes da gravação.

Verificação e reversão

Conferir kubectl get hpa/top, gerar carga sintética limitada, observar aumento e redução, registrar readiness e ausência de erros. Reverter thresholds/políticas por PR e rollout controlado se houver oscilação. Não usar dados reais nem tráfego irrestrito para demonstração.

Fonte: hpa.yaml.

ADR-003 - Observabilidade compartilhada no Learner Lab

Status: estado implementado documentado nesta revisão.

Contexto

O requisito permite escolha livre de ferramenta. O cluster precisa expor sinais técnicos, logs e traces sem abrir publicamente o management. O EBS CSI ficou CREATE_FAILED porque não conseguia obter credenciais; simulação IAM negou alterar trust/permissões da LabRole.

Decisão registrada

Prometheus/Grafana para métricas; Alloy/Loki para JSON; Alloy/Tempo para tracing; Blackbox para a sondagem pública; logs Lambda/Gateway no CloudWatch. A stack vive em observability. Somente Grafana é roteado publicamente pelo Gateway/NLB interno com login obrigatório. Dois dashboards versionados na pasta FIAP substituem os genéricos; a visão geral é a home.

No Lab, usar emptyDir e retirar o add-on EBS defeituoso após plano controlado que confirmou PVCs Pending sem volume. Dashboards/configurações são reprovisionados do Git; histórico e alterações manuais no Grafana podem ser perdidos quando os pods forem recriados.

Consequências e limites

A decisão viabiliza a demonstração, sem garantia de retenção durável. Não se declara que uma configuração de retenção em horas/dias garante sobrevivência a recriação de pod. Produção comercial deve reavaliar armazenamento e IAM dedicado. A Lambda tem tracing opcional; o caminho privado/exportação não foi demonstrado nesta revisão.

O deploy reconcilia ServiceMonitor da aplicação, pois o primeiro deploy pode ocorrer antes do CRD, e aplica a Probe após o Blackbox. Dados de negócio exigem instrumentação/agregação própria; gráficos de HTTP/JVM não equivalem a volume diário e tempo médio de cada etapa da OS.

Verificação

Workflow de publicação, login/banco Grafana HTTP 200, dois dashboards, séries API/uptime, logs JSON e traces reais. Testar alertas específicos de OS e a entrega/notificação antes de declarar encerrada a conformidade de monitoramento.

Matriz de atendimento - Fase 3

Revisão: 31/10/2026 (BRT). Conferência baseada no enunciado fornecido, no código, nos workflows e em testes pelo API Gateway. Esta matriz registra a validação anterior ao encerramento do Learner Lab solicitado pelo responsável.

Requisitos e evidências

Requisito	Resultado e evidência
API Gateway e rotas protegidas	HTTPS, integração privada, Swagger/OpenAPI e Authorizer validados no deploy completo
Autenticação serverless por CPF	CPF válido de CUSTOMER ativo retorna 200/JWT; CPF inválido retorna 400; cliente sintético inativo retorna 401 no teste público final. O cliente exclusivo de validação foi removido após o teste
JWT e RBAC	CUSTOMER consulta suas OS com 200; ausência de token 401; token malformado/alterado e acesso operacional indevido 403. Matriz do Authorizer e smoke do deploy atual
Ciclo de OS	As 12 requisições da coleção passaram: abertura 201, diagnóstico 200, orçamento 201, login CPF 200, consulta 200, erros 401/403, aprovação 200, conclusão 200, entrega 200 e leitura final DELIVERED 200
Quatro repositórios e proteção de main	Aplicação, Lambda, Kubernetes e banco separados; rulesets exigem PR; READMEs contêm propósito, stack, diagramas, execução/deploy e links da API
CI/CD e deploy por branch	Workflows validam PRs e publicam automaticamente pushes em homolog/production. Execução real foi comprovada em homolog; não se declara execução de produção neste Lab
RDS PostgreSQL gerenciado	Consumido pela aplicação e pela autenticação, com cliente ativo/inativo verificado pelo fluxo público. Apply do banco
Kubernetes escalável	EKS, Metrics Server e HPA 1-6 pods. Teste aprovado: 704 tentativas de carga, zero erros, 2 -> 5 pods Ready -> 2. Revalidação após os indicadores de negócio: 654 tentativas, zero erros, 2 -> 5 -> 2 pods, p95 cliente 443ms
Terraform e containers	Estados separados, planos com lock, EKS/Gateway/RDS/Lambdas provisionados; Dockerfile da aplicação. Lambdas usam ZIP Go; os repositórios Terraform não exigem container de runtime
CPU, memória e health/uptime	Dois dashboards FIAP; memória em MB, CPU em mCPU, probes públicas e readiness/liveness. Histórico depende da janela e da vida dos pods da stack
Latência, tráfego e erros HTTP	Histogramas e contadores publicados após corrigir limites de Timer para nanossegundos; p95 e taxa por rota consultáveis. Sem tráfego recente, p95 não possui amostra; não se substitui por um zero fictício
Volume diário de OS	Gauge persistido `workshop_work_orders_opened_today`, consultado do PostgreSQL a cada 30s; painel usa max entre réplicas. Dia do banco em UTC; não soma o mesmo agregado por pod
Duração média por status	`workshop_work_order_stage_duration_seconds`: DIAGNOSIS (Diagnóstico), IN_PROGRESS (Execução), COMPLETED até DELIVERED (Finalização). Considera etapas encerradas do histórico e exclui OS canceladas; sem etapa encerrada, mantém ausência de amostra
Falhas de integrações	Contadores dedicados de sucesso/falha para SMTP e coleta dos indicadores no PostgreSQL, com painel por integração. O SMTP do sandbox apresentou falhas reais; a OS continuou normalmente e o sinal foi registrado
Alertas de OS	Regras `WorkOrderProcessingFailure` (5xx em rota de OS) e `WorkOrderNotificationFailure` (SMTP). 4xx esperados não disparam falha de processamento. Teste promtool comprova firing/resolved; auditoria observou alerta SMTP real em firing

Requisito	Resultado e evidência
Logs e traces correlacionados	JSON/MDC com requestId e traceId; log de conclusão sem CPF, token, senha ou corpo. Auditoria encontrou traces reais da aplicação no Tempo e logs com os mesmos identificadores, sem exportar seus valores
Arquitetura e modelo de dados	Componentes, sequências de CPF/abertura de OS, RFCs, ADRs, ER e justificativa formal do PostgreSQL publicados neste diretório, com PDFs e diagramas separados
Vídeo <=15 minutos	Demonstração Fase 3, 13min29s. Acesso/duração conferidos nos metadados; o autor confirmou a demonstração de logs e traces. Não se apresenta essa confirmação como revisão independente do vídeo
PDF e avaliador	PDF único do Portal com quatro repositórios, vídeo, documentação e confirmação de `soat-architecture`, reconfirmado via API nos quatro repositórios

Resultados HTTP do teste público final e resumo persistido da carga/HPA final, sem identificadores ou credenciais.

Evidência final da observabilidade

Auditoria com verificações obrigatórias: volume diário disponível, três séries de duração, contadores de integração/processamento, duas regras de OS e traces com logs correspondentes. Resultado: `phase3_observability_validation PASSED`.

A auditoria observou `WorkOrderNotificationFailure=firing` após falhas SMTP reais; a conferência posterior confirmou a resolução para `inactive` e passou novamente em todas as verificações. `WorkOrderProcessingFailure` permaneceu inativo durante o ciclo da OS bem-sucedido. As regras e o teste reprodutível de disparo/resolução estão em observability/rules e observability/tests. Alertas são consultáveis no Grafana/Prometheus/Alertmanager; não foi configurada entrega externa por e-mail ou chat.

Validação de código

- Aplicação: `./mvnw verify` completo em Java 21, inclusive após consolidar homolog em main;

PR 90 com CI aprovado.

- PostgreSQL 16: consulta dos agregados verificada com datas conhecidas, três durações,

abertura diária e exclusão de cancelamentos/etapas ainda abertas.

- Lambda: testes Go e `go vet`, além dos checks do PR de consolidação.

- Terraform: formatação/validação de PRs e planos de remoção limitados aos recursos FIAP.

Limites conhecidos do ambiente acadêmico

- Observabilidade usa emptyDir: histórico de métricas/logs/traces é volátil. EBS CSI

não foi novamente aplicado após os bloqueios IAM do Academy.

- Os 503 da carga antiga não foram reproduzidos no teste aprovado de 704 tentativas;

isso comprova aquela janela e carga limitada, sem afirmar capacidade ilimitada ou uma causa única.

- O SMTP é sandbox e pode recusar entregas por limite/indisponibilidade. Essas falhas

são sinalizadas e não revertem o atendimento; o teste do ciclo não promete entrega de e-mail.

- Homolog foi executado; production possui automação configurada e não foi implantado

apenas para produzir evidência adicional ou aumentar custo no Lab.

- Os endpoints de runtime dependem dos recursos AWS. Após o destroy solicitado,

a avaliação deve usar o vídeo, código, diagramas e evidências dos workflows.

Encerramento após a validação

Registro completo. Todas as stacks foram removidas, com zero recursos gerenciados, e a verificação final com limpeza do bucket passou. Os endpoints de runtime e a massa deixaram de existir após o destroy solicitado; vídeo e repositórios permanecem disponíveis.

Roteiro simples - demonstração da Fase 3

Vídeo gravado: 13min29s. Este roteiro permanece como guia de apresentação e consulta; o conteúdo do vídeo ainda não foi revisado nesta atualização.

Meta de referência: 12 a 14 minutos. Limite do enunciado: 15 minutos. Consultar a matriz antes da submissão; não apresentar painel vazio como indicador atendido.

Preparação

- Abrir o diagrama de componentes, os quatro repositórios e uma execução de deploy concluída.

- Abrir Swagger, cliente HTTP e os dois dashboards do Grafana.

- Usar apenas dados sintéticos. Ocultar CPF, senha e JWT no enquadramento.

- Fazer o seed automático antes da gravação: seed_only, 6 OS. Importar no Postman

o ambiente Massa Pronta exportado pela Action; não cadastrar cliente, veículo ou peça manualmente. A carga é separada: load_only, 180 segundos, concorrência 2. Esses valores menores ainda precisam de validação de carga.

- Não executar carga junto com rollout. Não repetir seed para gerar pressão: cada lote permanece.

Acesso rápido: guia com links e Postman. A massa é preparada antes da gravação; no vídeo não é necessário mostrar cadastros de cliente, veículo ou peça.

1. Apresentação e arquitetura - 1 minuto

Fala sugerida: “Esta é a aplicação da oficina. Organizamos a solução em quatro repositórios: aplicação, autenticação serverless, infraestrutura Kubernetes e banco gerenciado.” Mostrar o diagrama e apontar Gateway, Lambda, EKS, PostgreSQL e observabilidade.

2. Pipeline e deploy - 2 minutos

Mostrar um PR, proteção da main, checks e uma execução concluída em homologação. Apontar teste, publicação da imagem, rollout e smoke pelo Gateway. Mostrar também os workflows dos outros três repositórios. Dizer que produção tem triggers configurados, mas sua execução ainda não está comprovada nesta homologação.

3. Autenticação e API protegida - 2 minutos

Fazer login com CPF sintético ativo sem revelar seu valor nem o JWT. Mostrar apenas HTTP 200. Consumir uma rota CUSTOMER com sucesso. Repetir sem token (401) e tentar uma rota operacional como CUSTOMER (403). Explicar que a Lambda verifica cadastro/status e emite JWT.

4. Ordem de serviço - 2 minutos

Fala sugerida: “A massa sintética já foi criada automaticamente pelo seed antes da gravação. Agora vou usar esse cliente, veículo e peça para demonstrar apenas o ciclo da OS.”

No Postman, executar a collection curta: login, abertura, diagnóstico, orçamento, login CPF, consulta permitida, testes 401/403, aprovação, conclusão, entrega e consulta final. Como funcionário, abrir uma OS e mostrar sua consulta. Usar o lote preparado para mostrar Recebida, Diagnóstico, Aguardando aprovação, Em execução, Concluída e Entregue. Explicar o orçamento e a reserva de estoque, sem expor dados pessoais.

5. Monitoramento e escala - 3 minutos

No Grafana, abrir Visão geral e Infraestrutura. Mostrar uptime, CPU, memória e réplicas. Iniciar uma carga leve separada do seed, observar o HPA e sua recuperação. Não prometer atingir seis pods. Se ocorrer 503, interromper e registrar a falha; não esconder o resultado. Mostrar os painéis HTTP de tráfego, erros e latência, cuja coleta foi validada após a correção dos histogramas. O p95 precisa de requisições na janela selecionada. Volume diário de OS, tempo médio por etapa e falhas de integrações ainda dependem de implementação/validação.

6. Logs, traces e alertas - 2 minutos

Selecionar uma requisição de teste, mostrar log JSON e o trace correspondente no Tempo. Demonstrar alerta de falha de OS disparando e resolvendo. Esses exemplos precisam estar validados antes da gravação final: logs com traceId, sozinhos, não comprovam trace exportado.

7. Documentação e encerramento - 1 minuto

Mostrar componentes, as duas sequências, ER, RFCs e ADRs nos arquivos separados. Encerrar com links dos quatro repositórios e confirmação de acesso do avaliador.

Depois da gravação

Publicar no YouTube/Vimeo como público ou não listado, conferir acesso e duração de até 15 minutos e incluir o link na folha de entrega. O Portal pede um PDF único com links dos repositórios, vídeo, documentação e confirmação do avaliador. Os arquivos separados são material complementar; não substituem o vídeo nem o PDF único.

Links dos documentos e fala sugerida

Não precisa ler cada arquivo inteiro no vídeo. Abra o documento, mostre o trecho indicado por 10 a 20 segundos e use estas frases como apoio, adaptando para o que está demonstrado.

Documento	O que mostrar / falar
Comece aqui	“Este é o índice de execução: reúne os acessos, a collection curta, o seed e as evidências das pipelines.”
Massa pronta	“A massa foi criada automaticamente antes da gravação. Cliente, veículo e peça já existem; no Postman vou demonstrar somente login e o ciclo da OS.”
Arquitetura e diagrama de componentes	“O Gateway é a entrada HTTPS. A Lambda autentica o cliente e autoriza o JWT; a aplicação está no EKS e persiste no PostgreSQL gerenciado. O Grafana reúne métricas e logs.”
Sequências, autenticação e abertura de OS	“Aqui mostramos a ordem das chamadas: CPF e token primeiro; depois o acesso autorizado e a transação de abertura da OS.”
Modelo de dados e ER	“Cliente e veículo se relacionam às OS; orçamento, itens e estoque mantêm a consistência do atendimento. O documento explica os relacionamentos e a evolução do modelo.”
RFC-001: nuvem	“A escolha de AWS e serviços gerenciados reduz a operação manual no laboratório. O documento registra alternativas e limitações do Learner Lab.”
RFC-002: PostgreSQL	“O banco relacional foi escolhido pelas transações e integridade entre OS, orçamento e estoque. A evolução do schema também está documentada.”
RFC-003: autenticação	“A função valida CPF e cadastro ativo, emite JWT e o Authorizer protege as rotas. A aplicação aplica RBAC; CPF sozinho é uma limitação deste fluxo acadêmico.”
ADR-001: comunicação	“Usamos HTTP síncrono no atendimento, com responsabilidades separadas para Gateway, autenticação e aplicação.”

Documento	O que mostrar / falar
ADR-002: HPA	“O HPA muda a quantidade de pods conforme CPU e memória. Isso é diferente de aumentar o número de nós.”
ADR-003: observabilidade	“A stack é compartilhada e há somente dois dashboards. No Lab o histórico é efêmero; um reinício pode reduzir a janela de dados disponível.”
Matriz de validação	“Esta matriz distingue requisito comprovado, implementação e pendência. Uma pipeline verde não comprova sozinha todos os requisitos funcionais.”
Investigação de 503	“Registramos as falhas de carga e as hipóteses sem esconder resultados. Um seed aprovado não comprova estabilidade sob carga.”
Folha de entrega	“Ao concluir a gravação, este documento reúne os quatro repositórios, o vídeo, a documentação e o acesso do avaliador.”

Links das pipelines dos quatro repositórios

Repositório / pipeline	O que faz	Fala sugerida
App: CI	Build e testes Java/Quarkus, qualidade/cobertura, segurança e validação dos manifestos	“O PR passa pelos testes e verificações antes da entrega.”
App: deploy	Em homolog/production, valida, publica imagem no ECR, aplica no cluster, espera readiness e testa Gateway/Authorizer/docs	“Aqui a imagem é identificada pelo commit e o rollout termina com testes pelo endereço público.”
Lambda: CI	Formatação, go vet, testes unitários e de integração Go	“A função tem verificação própria, independente da aplicação.”
Lambda: Terraform / deploy	Compila autenticação e Authorizer, valida/plana/aplica Terraform e reconcilia o Gateway	“A publicação da função também atualiza os contratos usados pelo Gateway.”
Lambda: validação pelo Gateway	Exercita login CPF, JWT e negativas de autorização via API pública	“Validamos autorização no caminho real, com respostas permitidas e negadas.”
Kubernetes: Terraform	Valida Terraform e testes de convergência; aplica infraestrutura nas branches de ambiente	“O cluster e seus recursos de infraestrutura são versionados neste repositório.”
Kubernetes: reconciliação Gateway	Consome contratos publicados e reconcilia as integrações do Gateway	“Este passo mantém aplicação e autenticação conectadas ao Gateway.”
Kubernetes: observabilidade	Instala/reconcilia a stack de monitoramento e publica a rota privada do Grafana	“O monitoramento é provisionado separadamente da imagem da aplicação.”
Kubernetes: dashboards	Atualiza somente os ConfigMaps dos dois dashboards e verifica a configuração salva	“Podemos melhorar os painéis sem reinstalar a stack nem executar Terraform apply.”
Banco: Terraform	Formatação/validação, plano e apply do RDS e da rede do banco por ambiente	“O banco gerenciado tem infraestrutura e pipeline próprias. As migrations da aplicação são outro mecanismo.”

Exemplos prontos para abrir na gravação

- PR aprovado com a massa exportável.
- Deploy da aplicação aprovado com smokes.
- Deploy da Lambda aprovado.
- Apply do banco aprovado.

- Seed aprovado desta massa.

Mostrar execução concluída e sua branch/commit. Não disparar apply de banco ou cluster somente para preencher tempo de vídeo. Os exemplos são de homologação; não afirmar execução comprovada de produção quando houver apenas configuração do gatilho.

Action para demonstrar aumento de réplicas

Abrir Homolog - seed and HPA simulation.

1. Run workflow, branch main.

2. operation=load_only: a massa já está pronta; não precisa criar outro lote.

3. duration_seconds=180, concurrency=2. work_orders é ignorado nesse modo.

4. Abrir o dashboard Infraestrutura

e acompanhar CPU, memória e réplicas desejadas/disponíveis. Depois esperar a recuperação.

Fala: “Esta action gera logins válidos com carga gradual. O HPA observa o consumo e decide se precisa criar mais pods. Ao final, a carga para e acompanhamos a redução. Não estamos alterando réplicas manualmente.”

O perfil leve tem limite global de quatro requisições/s. Aumentar pods não é garantido com qualquer carga: depende da utilização e do estado inicial. Não elevar agressivamente os valores para forçar uma cena. Se ocorrer 503 ou falha no orçamento de erros, interromper a demonstração e consultar o resumo. Não apresentar a execução reprovada como sucesso.

Como ler os painéis durante a apresentação

- Memória está em MB e CPU em millicores. Réplicas desejadas mostram a decisão do HPA;

disponíveis indicam pods efetivamente prontos.

- A seção de logs começa recolhida. Abrir pela seta, ler horário/nível/mensagem e clicar

na linha quando precisar dos campos técnicos. Explore mantém acesso à consulta original.

- Os histogramas HTTP foram corrigidos e a coleta foi validada. Se não houver tráfego

recente, gerar poucas chamadas da collection e aguardar o intervalo de coleta; não chamar um p95 vazio de zero. Erros rejeitados no Gateway/Authorizer são outro salto e não devem ser confundidos com respostas 5xx emitidas pela aplicação.

Evidências disponíveis ao concluir a gravação

- Vídeo final, 13min29s.

- Deploy completo e smokes.

- Auditoria obrigatória de métricas, regras e correlação log/trace.

- Carga de 704 tentativas sem erro, HPA 2 -> 5 -> 2.

Ao explicar os novos painéis: “O volume diário vem do banco, e não da memória do pod. As durações usam o histórico de etapas encerradas; cancelamentos são excluídos. Falhas SMTP aparecem como falhas da integração e geram alerta sem desfazer a OS.” O ambiente é encerrado depois da validação, conforme solicitado; os links de runtime dependem de AWS ativa.

Revalidação da versão final: 654 tentativas sem erro e HPA 2 -> 5 -> 2. O alerta SMTP foi observado disparando e depois resolvendo, sem derrubar o ciclo da OS.

Entrega no Portal - Fase 3

PDF único para enviar à instituição.

O PDF reúne os links exigidos pelo enunciado: quatro repositórios, vídeo de até 15 minutos, documentação arquitetural e confirmação do avaliador. Os documentos separados e o dossiê continuam disponíveis para consulta; não é necessário anexar um ZIP.

Vídeo

FIAP - 16SOAT - Demonstração Fase 3, **13min29s**. Acesso e duração conferidos nos metadados do YouTube. O autor confirmou que mostrou logs e traces em execução. A matriz separa essa confirmação da evidência técnica dos workflows.

Repositórios

1. oficina-lambda-auth
2. oficina-infra-k8s
3. oficina-infra-db
4. oficina-app

Acessos e comprovação

- Documentos, diagramas, RFCs e ADRs.
- Validação final e limites do Lab.
- Roteiro com documentos, pipelines e demonstração de réplicas.
- Coleção de 12 requisições e massas prontas.
- PR de implementação com CI aprovado.
- Deploy completo aprovado.
- Auditoria final de observabilidade.
- Seed/carga/HPA.

`soat-architecture` foi reconfirmado nos quatro repositórios via API do GitHub. A homologação foi validada antes do encerramento do Learner Lab solicitado pelo responsável. Swagger e Grafana dependem de infraestrutura ativa; vídeo, código, documentos e evidências permanecem no GitHub/YouTube.

Regeneração

Editar entrega-portal.json e executar `python3 docs/fase3/gerar-entrega-portal.py` com ReportLab instalado. Não incluir credenciais, tokens ou senhas no PDF.

Ambiente encerrado

O destroy foi concluído após os testes. Comprovação e detalhes. Finalização aprovada. API, Swagger e Grafana estão desligados; enviar o PDF, o vídeo e os links dos repositórios conforme o enunciado.